Arc42 — Plataforma de Venda de Tickets

> Documentação de Arquitetura de Software seguindo o template Arc42 (11 seções) Projeto: PJBL — Arquitetura de Soluções em Nuvem Equipe: Arthur Kenji Kussaba · Frederico Salvatti Cavalcante · Eduardo Antonio Babinski Pedroso Data: 2026-06-15

Links do Projeto

Artefato	URL
Repositório GitHub	https://github.com/akkenji000/Cloud-pjbl-plataforma-tickets
Docker Hub — MS Catálogo	https://hub.docker.com/r/akknj0000/ms-catalogo-eventos
Docker Hub — MS Pedidos	https://hub.docker.com/r/akknj0000/ms-pedidos
Docker Hub — BFF	https://hub.docker.com/r/akknj0000/bff-nodejs
Docker Hub — API Gateway	https://hub.docker.com/r/akknj0000/api-gateway
Docker Hub — Frontend	https://hub.docker.com/r/akknj0000/frontend-spa
Swagger BFF (local)	http://localhost:3000/api/docs
Swagger MS Catálogo (local)	http://localhost:5001/swagger
Swagger MS Pedidos (local)	http://localhost:5002/swagger
Azure Function (produção)	https://pjbl-taxas-function-dhbcg8bjhtbafya5.canadacentral-01.azurewebsites.net
Vídeo YouTube	*(link a ser adicionado após gravação)*

---

1. Introdução e Objetivos

1.1 Escopo do Sistema

A **Plataforma de Venda de Tickets** é um sistema distribuído baseado em microsserviços para gerenciamento e comercialização de ingressos para eventos culturais e esportivos. O sistema permite que usuários consultem o catálogo de eventos, realizem pedidos de ingressos e recebam valores de taxa de conveniência calculados dinamicamente por uma Azure Function serverless.

1.2 Objetivos de Qualidade

Prioridade	Atributo de Qualidade	Descrição
1	Escalabilidade	Cada microsserviço pode escalar independentemente conforme demanda
2	Manutenibilidade	Clean Architecture + Vertical Slice facilitam evolução por feature
3	Resiliência	BFF usa `Promise.allSettled()` para chamadas paralelas com fallback
4	Testabilidade	Domínio isolado de infraestrutura; testes unitários e de arquitetura
5	Portabilidade	Todos os serviços containerizados via Docker, imagens no Docker Hub

1.3 Stakeholders

Stakeholder	Papel	Interesse
Usuário Final	Comprador de ingressos	Interface fluida e preços corretos
Administrador	Gestor da plataforma	CRUD de eventos e gestão de pedidos
Equipe de Desenvolvimento	Time PJBL	Código limpo, testável e bem documentado
Professor/Avaliador	Avaliador acadêmico	Conformidade com requisitos arquiteturais

---

2. Restrições de Arquitetura

2.1 Restrições Técnicas

- **Runtime .NET**: SDK 10.0 para microsserviços C# (ASP.NET Core 10)
- **Azure Functions**: Runtime net8.0 Isolated Worker, deployada diretamente no Azure Canada Central (não containerizada — ver ADR-005)
- **BFF**: Node.js 20 + NestJS (módulos de features, Vertical Slice)
- **Bancos de Dados**: MongoDB Atlas M0 FREE (eventos) e Azure SQL Basic 5 DTUs (pedidos)
- **Containerização**: Docker + Docker Compose obrigatórios; imagens publicadas no Docker Hub (`akknj0000/`)

2.2 Restrições Organizacionais

- Entrega até 15/06/2026 às 23:59
- Projeto acadêmico — sem SLA de produção; `EnsureCreated()` substitui migrations EF Core
- Repositório monorepo público no GitHub: https://github.com/akkenji000/Cloud-pjbl-plataforma-tickets

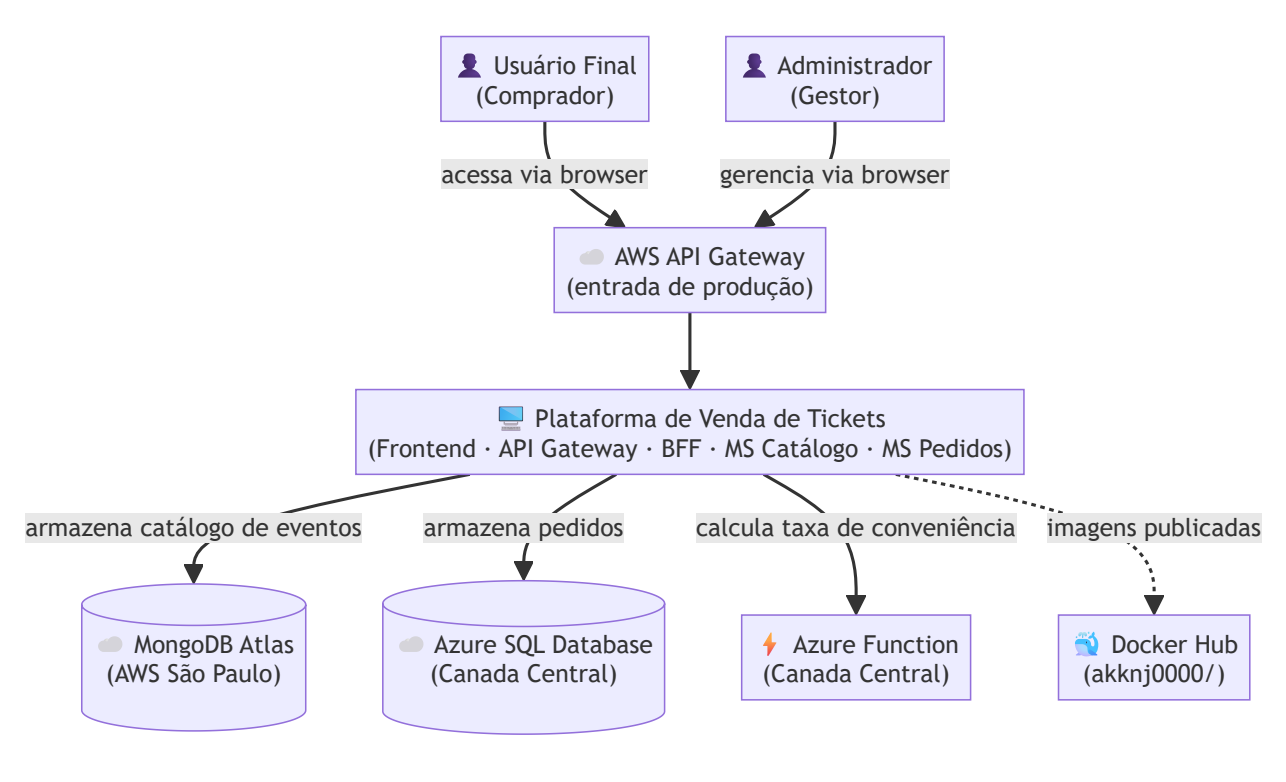
2.3 Convenções e Padrões

- Clean Architecture: isolamento Domain → Application → Infrastructure → API
- Vertical Slice: código agrupado por feature (`CreateEvento` , `GetEvento` , etc.)
- Persistência Poliglota: cada MS possui seu próprio banco de dados
- Sem compartilhamento de banco entre microsserviços

---

3. Contexto e Escopo do Sistema

3.1 Diagrama de Contexto (C4 Nível 1)


Usuário Final (Comprador)
Administrador (Gestor)
acessa via browser
gerencia via browser
AWS API Gateway (entrada de produção)
Plataforma de Venda de Tickets (Frontend · API Gateway · BFF · MS Catálogo · MS Pedidos)
armazena catálogo de eventos
armazena pedidos
calcula taxa de conveniência
imagens publicadas
MongoDB Atlas (AWS São Paulo)
Azure SQL Database (Canada Central)
Azure Function (Canada Central)
Docker Hub (akknj0000/)

3.2 Contexto de Negócio

Parceiro Externo	Entrada	Saída
Usuário Final	Acessa via browser → AWS API Gateway	Visualiza eventos e faz pedidos
Administrador	CRUD via browser → AWS API Gateway	Respostas JSON com dados atualizados
AWS API Gateway	Requisições HTTP do browser	Roteamento para o BFF interno
MongoDB Atlas (AWS SP)	Operações do BFF → MS Catálogo	Persistência de catálogo de eventos
Azure SQL Database (Canada)	Operações do BFF → MS Pedidos	Persistência de pedidos e transações
Azure Function (Canada)	Chamada HTTPS do BFF (userId + preco)	Taxa de conveniência calculada
Docker Hub	—	Registro e distribuição das 5 imagens

3.3 Architecture Communication Canvas

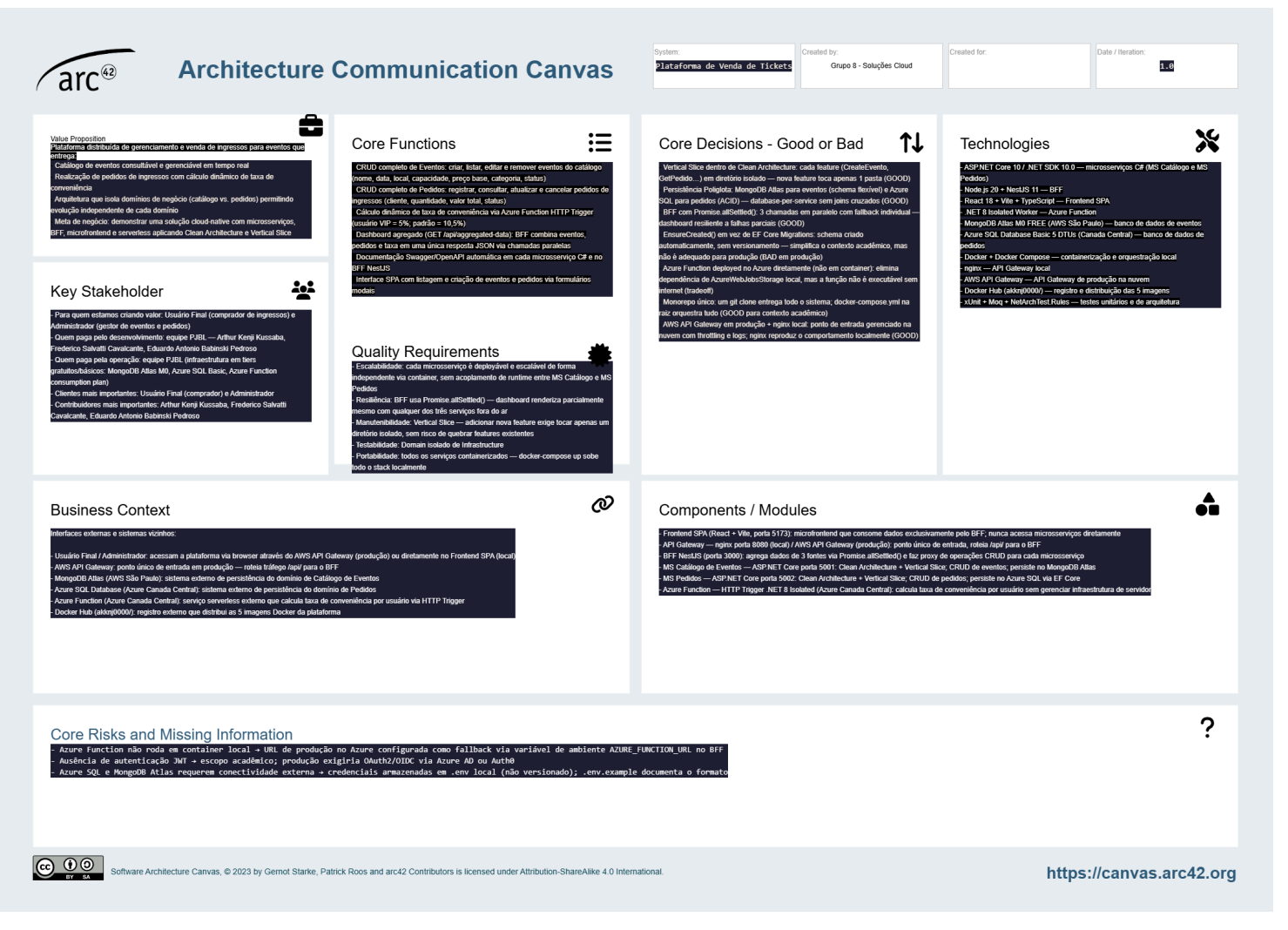

Architecture Communication Canvas
Value Proposition
Core Functions
Core Decisions - Good or Bad
Technologies
Key Stakeholder
Quality Requirements
Business Context
Components / Modules
Core Risks and Missing Information
https://canvas.arc42.org

Figura 0: Software Architecture Canvas preenchido — visão condensada da arquitetura da plataforma.

3.4 Frontend Funcionando

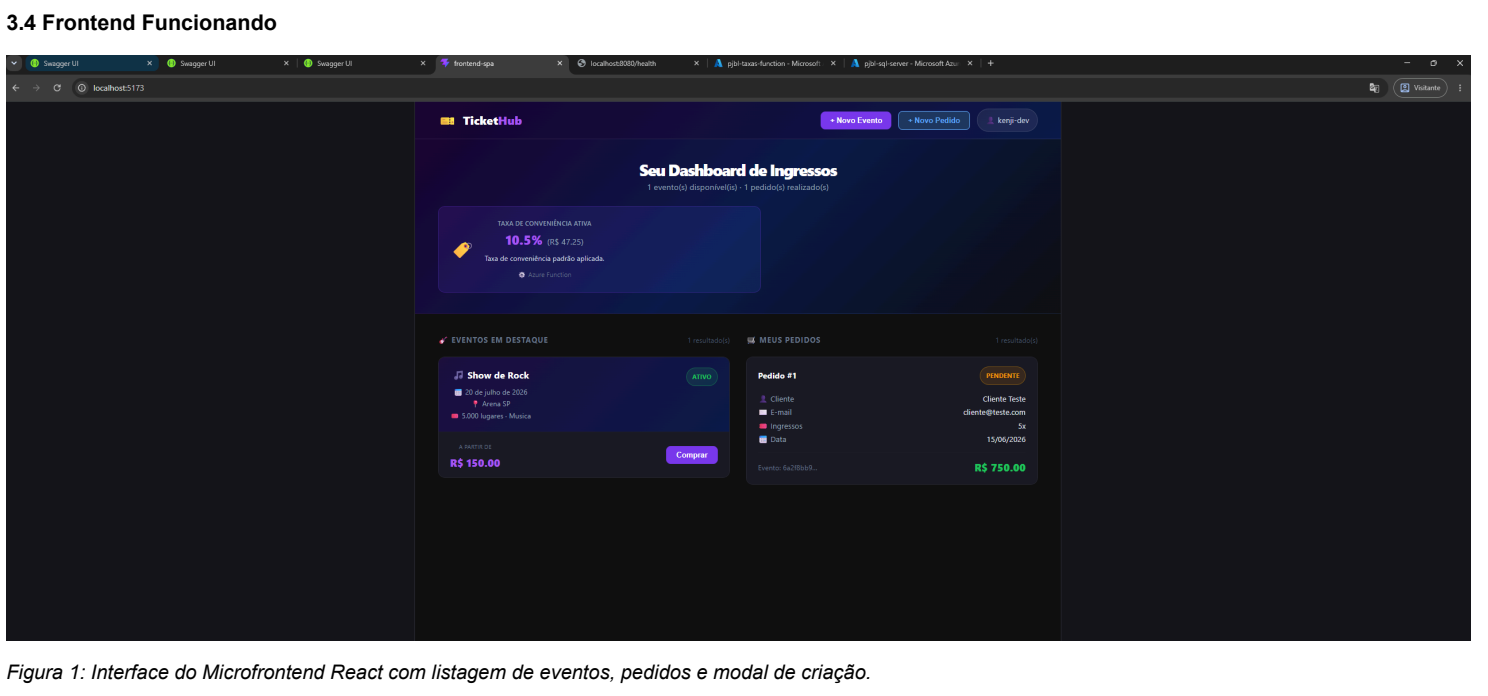

TicketHub
+ Novo Evento
+ Novo Pedido
Seu Dashboard de Ingressos
10.5%
Eventos em destaque
Show de Rock
R$ 150.00
Comprar
Meus pedidos
Pedido #1
R$ 750.00

Figura 1: Interface do Microfrontend React com listagem de eventos, pedidos e modal de criação.

---

4. Estratégia de Solução

4.1 Decisões Arquiteturais Fundamentais

Decisão	Escolha	Justificativa
Arquitetura	Microsserviços	Isolamento de domínios, escalabilidade independente
Padrão interno	Clean Architecture + Vertical Slice	Coesão por feature, testabilidade do domínio
Frontend	Microfrontend (SPA)	Desacoplamento do backend, deploy independente
API Gateway	AWS API Gateway (nuvem) / nginx (local)	Centraliza entrada, roteia para BFF, abstrai topologia interna
Agregação de API	BFF (Backend for Frontend)	Reduz chattiness do frontend, oculta complexidade interna
Cálculo serverless	Azure Function (HTTP Trigger)	Isolamento de lógica de taxas, paradigma serverless
Banco MS Eventos	MongoDB Atlas	Flexibilidade de schema para entidades de eventos
Banco MS Pedidos	Azure SQL + EF Core	Consistência transacional para dados financeiros

4.2 Padrão de Resiliência

O BFF utiliza `Promise.allSettled()` para chamar os três serviços em paralelo. Em caso de falha de qualquer serviço, o BFF retorna dados de fallback em vez de propagar erros ao frontend.

4.3 Estilos Arquiteturais Aplicados

4.3.1 Serverless

A **Azure Function** (`CalcularTaxaFunction`) é o componente serverless do sistema. Implementada no modelo Isolated Worker do Azure Functions v4, ela responde a um **HTTP Trigger** (`GET /api/calcular-taxa`) e calcula a taxa de conveniência por usuário sem gerenciar infraestrutura de servidor. O ciclo de vida da função é gerenciado pelo runtime do Azure Functions.

4.3.2 Microfrontend

O **frontend-spa** é uma SPA em React/Vite que consome dados **exclusivamente pelo BFF** (nunca diretamente pelos microsserviços), respeitando o contrato de isolamento de camadas.

4.3.3 Backend for Frontend (BFF) + Microservices + Database per Service

BFF: O **bff-nodejs** (NestJS :3000) possui dois papéis:

1. **Agregação** (`GET /api/aggregated-data`): chama MS Catálogo, MS Pedidos e Azure Function em paralelo e retorna um único JSON consolidado.
2. **Proxy CRUD**: repassa operações de escrita/leitura para cada microsserviço.

Database per Service: MongoDB Atlas para eventos, Azure SQL para pedidos. Sem compartilhamento de banco.

4.3.4 API Gateway

O projeto utiliza **duas implementações de API Gateway**, conforme o ambiente:

- **Ambiente local (Docker Compose):** nginx (:8080) atua como gateway, roteando todo tráfego `/api/` para o BFF via `proxy_pass http://bff;` (sem trailing slash, preservando o prefixo completo da URI).
- **Ambiente de nuvem / produção: AWS API Gateway** substitui o nginx como ponto de entrada externo, oferecendo roteamento gerenciado, throttling, logs no CloudWatch e escalabilidade automática.

Em ambos os casos, clientes externos conhecem apenas o endpoint do gateway — a topologia interna dos microsserviços permanece abstraída.

Fluxo: `Frontend → AWS API Gateway (nuvem) / nginx :8080 (local) → BFF (:3000) → Microsserviços / Azure Function`

4.3.5 Arquitetura Orientada a Eventos (EDA)

A plataforma utiliza comunicação **síncrona HTTP**. O padrão EDA está presente de forma embrionária na Azure Function (orientada a eventos por natureza). A evolução natural seria publicar eventos como `PedidoCriado` em Azure Service Bus, consumidos por MS de notificação e atualização de vagas.

4.3.6 Clean Architecture

Cada microsserviço C# segue 4 camadas isoladas:

- **Domain**: entidades e interfaces de repositório — sem dependências externas
- **Application**: handlers de features (Vertical Slice)
- **Infrastructure**: implementações concretas (MongoDB, EF Core)
- **Controllers (API)**: recebem HTTP, delegam para Application

4.3.7 Vertical Slice

```
Application/Features/
  CreateEvento/   ← CreateEventoHandler + CreateEventoRequest + CreateEventoResponse
  GetEvento/      ← GetEventoHandler + GetEventoResponse
  UpdateEvento/   ← UpdateEventoHandler + UpdateEventoRequest
  DeleteEvento/   ← DeleteEventoHandler
```

4.3.8 Testes Unitários e de Arquitetura

Testes Unitários (xUnit + Moq): Validam handlers da camada Application com repositório mockado.

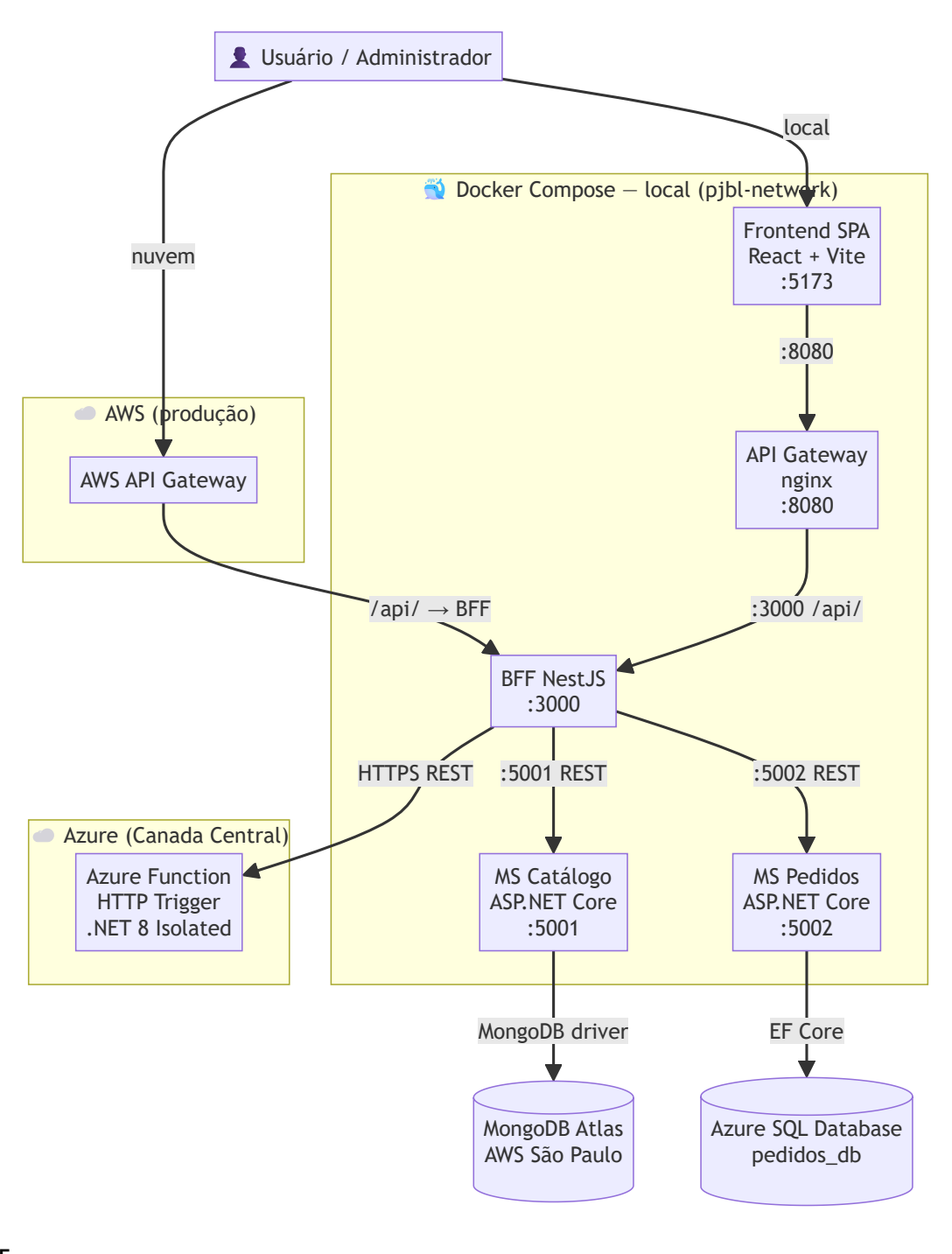
Testes de Arquitetura (NetArchTest.Rules): Verificam que Domain não depende de Infrastructure:

```
Types.InAssembly(assembly)
    .That().ResideInNamespace("..Domain..")
    .ShouldNot().HaveDependencyOn("..Infrastructure..")
    .GetResult().IsSuccessful // true
```

---

5. Visão de Building Blocks

5.1 Diagrama de Containers (C4 Nível 2)


Usuário / Administrador
nuvem
local
Docker Compose — local (pjbl-network)
Frontend SPA React + Vite :5173
:8080
API Gateway nginx :8080
:3000 /api/
AWS (produção)
AWS API Gateway
/api/ → BFF
BFF NestJS :3000
HTTPS REST
:5001 REST
:5002 REST
Azure (Canada Central)
Azure Function HTTP Trigger .NET 8 Isolated
MS Catálogo ASP.NET Core :5001
MS Pedidos ASP.NET Core :5002
MongoDB driver
EF Core
MongoDB Atlas AWS São Paulo
Azure SQL Database pedidos_db

5.2 Swagger — BFF

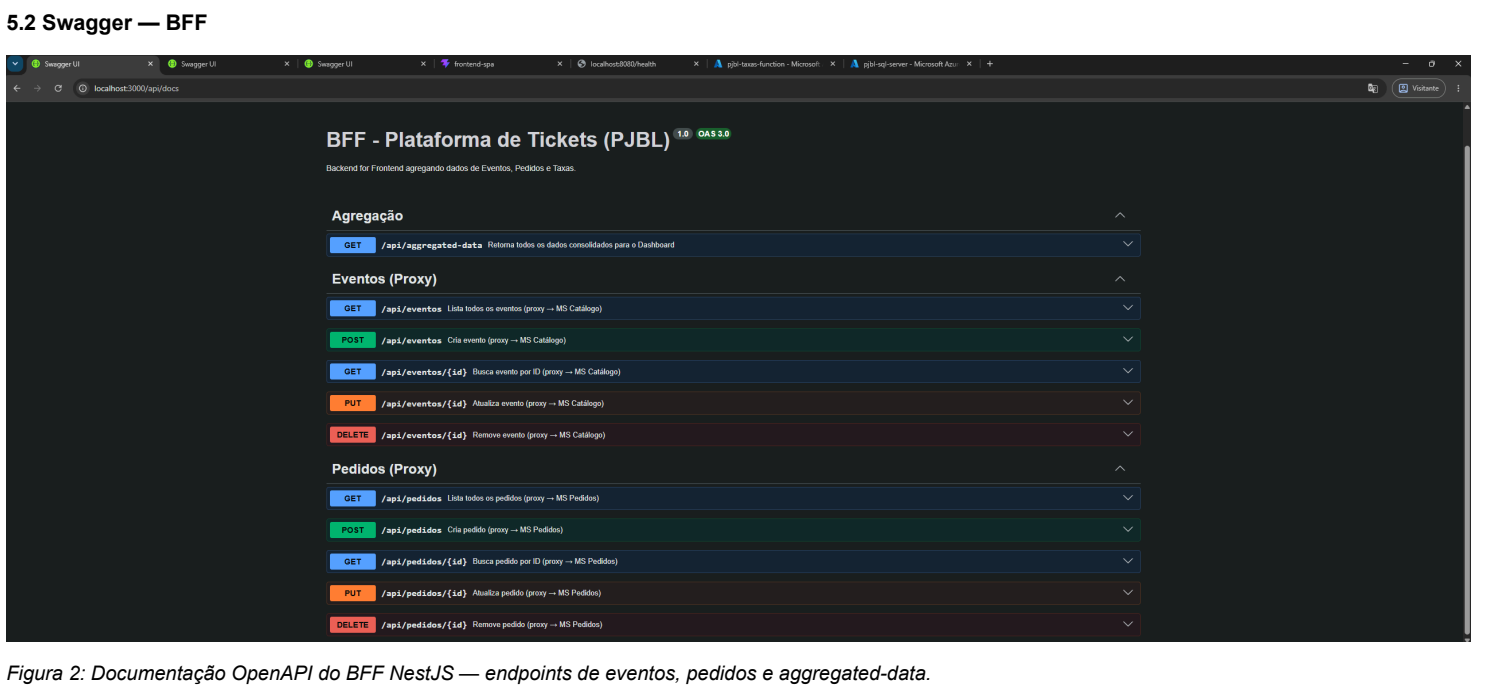

BFF - Plataforma de Tickets (PJBL)
Agregação
GET /api/aggregated-data
Eventos (Proxy)
GET /api/eventos
POST /api/eventos
GET /api/eventos/{id}
PUT /api/eventos/{id}
DELETE /api/eventos/{id}
Pedidos (Proxy)
GET /api/pedidos
POST /api/pedidos
GET /api/pedidos/{id}
PUT /api/pedidos/{id}
DELETE /api/pedidos/{id}

Figura 2: Documentação OpenAPI do BFF NestJS — endpoints de eventos, pedidos e aggregated-data.

5.3 Swagger — MS Catálogo de Eventos

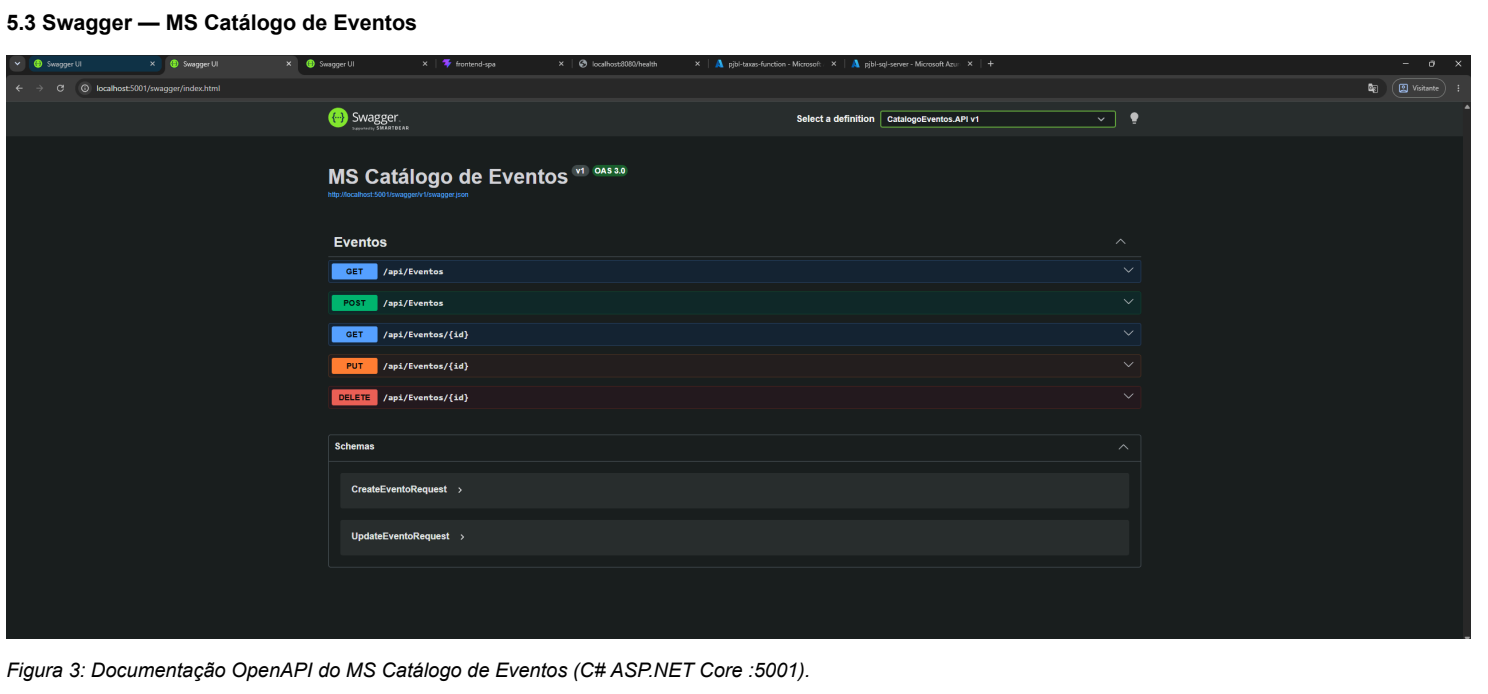

Swagger
MS Catálogo de Eventos
Eventos
GET /api/Eventos
POST /api/Eventos
GET /api/Eventos/{id}
PUT /api/Eventos/{id}
DELETE /api/Eventos/{id}
Schemas
CreateEventoRequest
UpdateEventoRequest

Figura 3: Documentação OpenAPI do MS Catálogo de Eventos (C# ASP.NET Core :5001).

5.4 Swagger — MS Pedidos

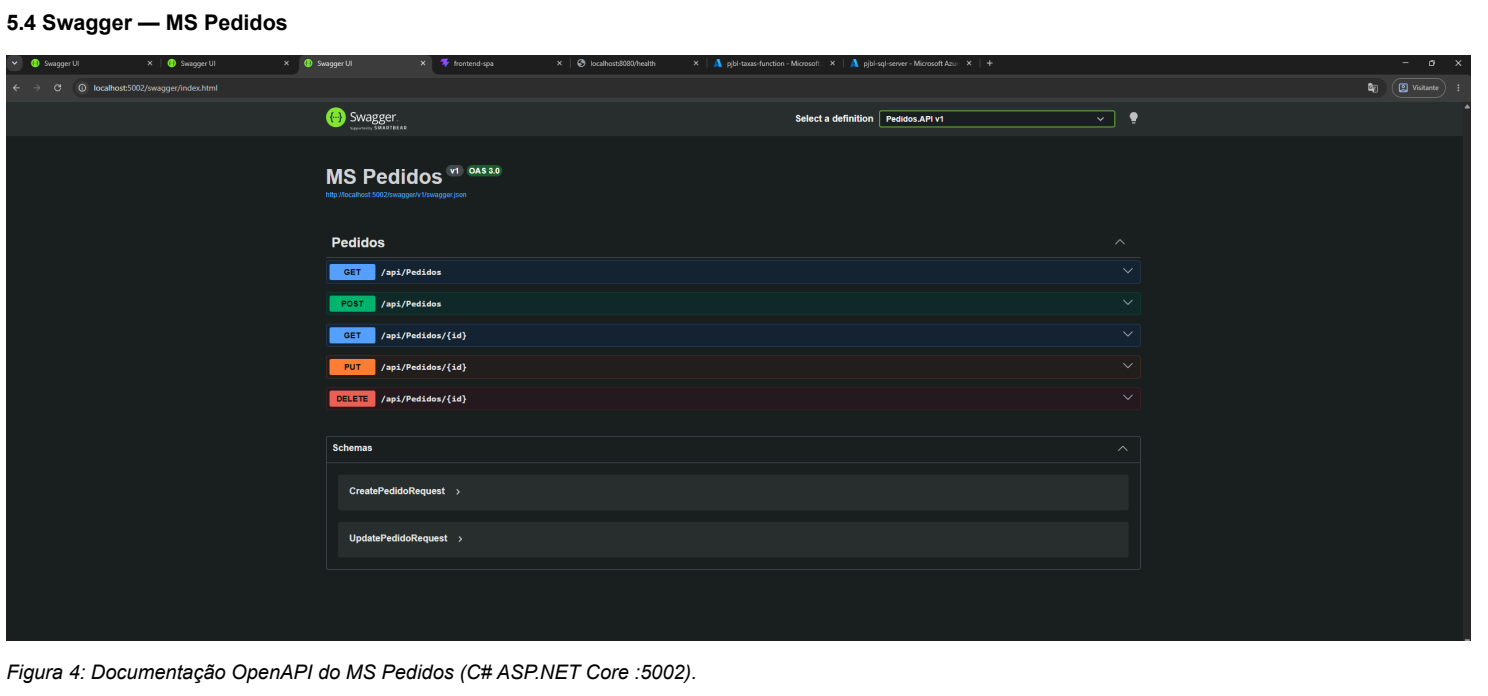
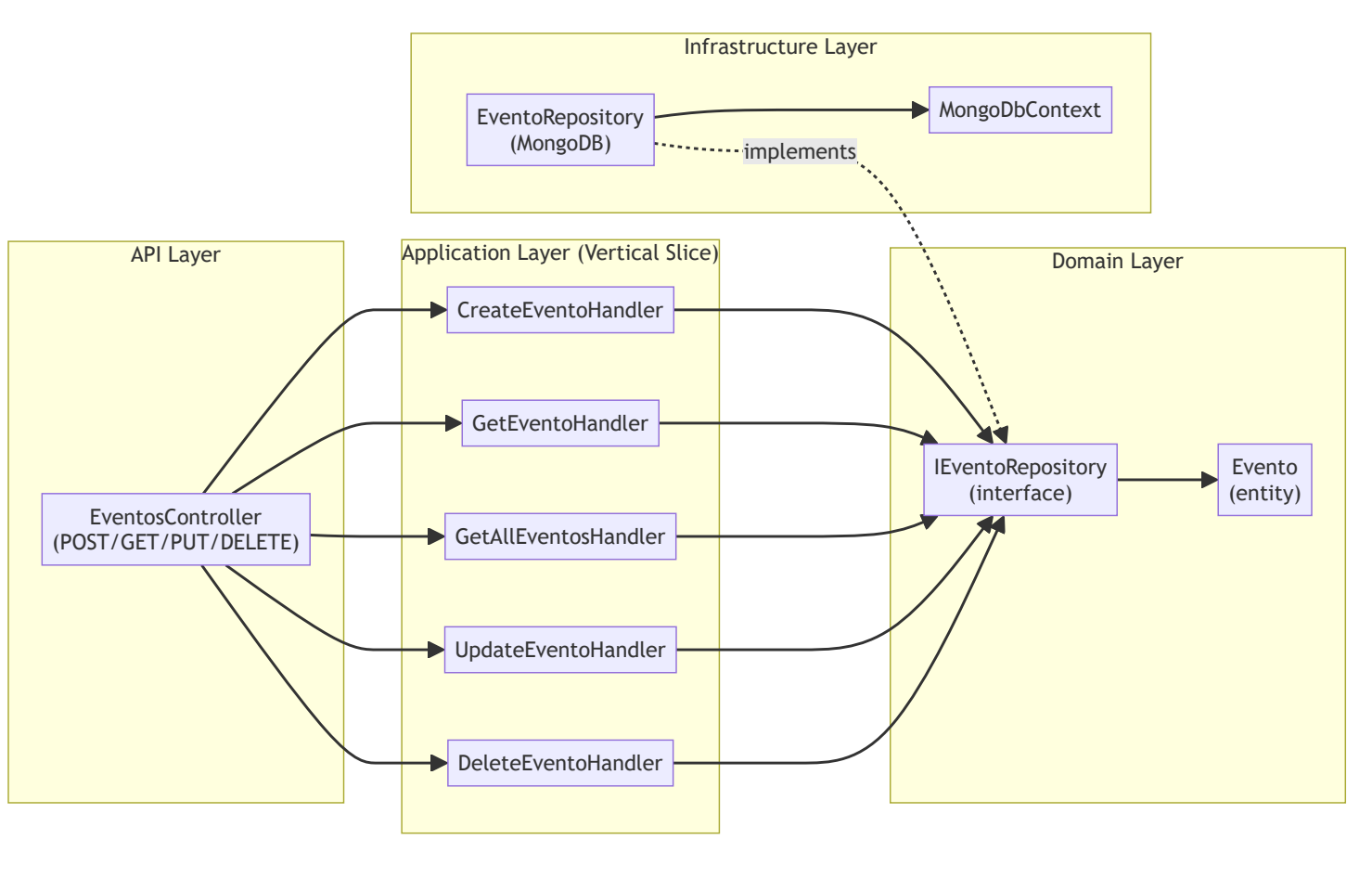
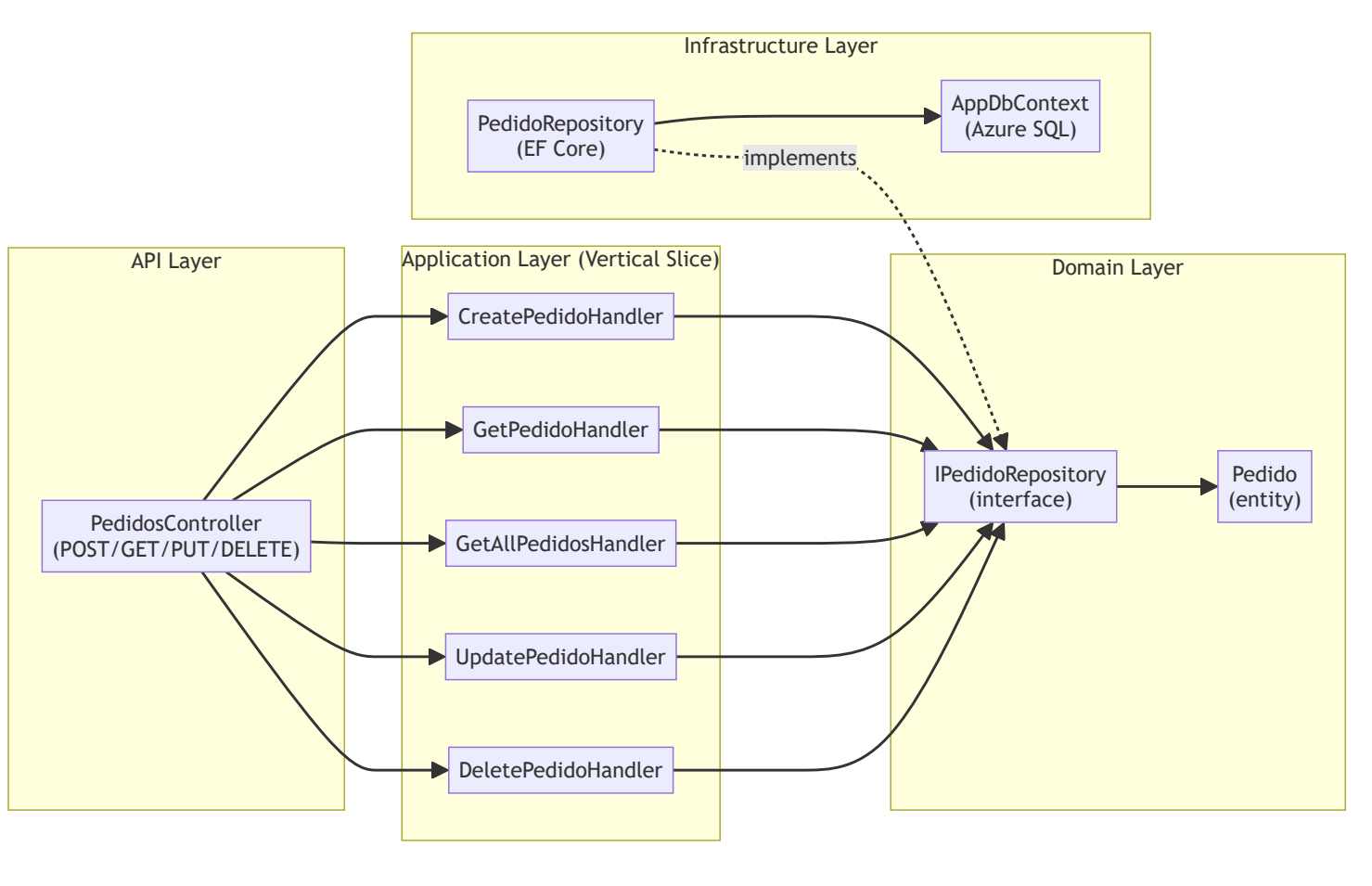

Swagger
MS Pedidos
Pedidos
GET /api/Pedidos
POST /api/Pedidos
GET /api/Pedidos/{id}
PUT /api/Pedidos/{id}
DELETE /api/Pedidos/{id}
Schemas
CreatePedidoRequest
UpdatePedidoRequest

Figura 4: Documentação OpenAPI do MS Pedidos (C# ASP.NET Core :5002).

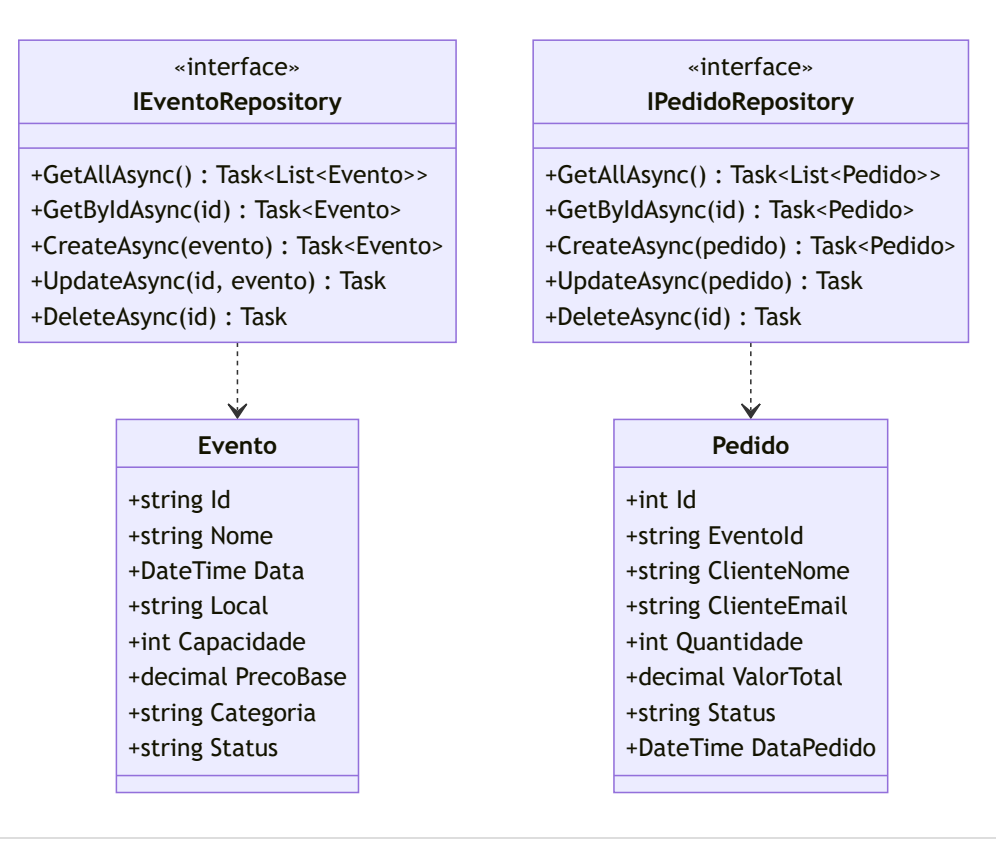
5.5 Diagrama de Componentes — MS Catálogo de Eventos (C4 Nível 3)


Infrastructure Layer
EventoRepository (MongoDB)
MongoDbContext
implements
API Layer
EventosController (POST/GET/PUT/DELETE)
Application Layer (Vertical Slice)
CreateEventoHandler
GetEventoHandler
GetAllEventosHandler
UpdateEventoHandler
DeleteEventoHandler
Domain Layer
IEventoRepository (interface)
Evento (entity)

5.6 Diagrama de Componentes — MS Pedidos (C4 Nível 3)


Infrastructure Layer
PedidoRepository (EF Core)
AppDbContext (Azure SQL)
implements
API Layer
PedidosController (POST/GET/PUT/DELETE)
Application Layer (Vertical Slice)
CreatePedidoHandler
GetPedidoHandler
GetAllPedidosHandler
UpdatePedidoHandler
DeletePedidoHandler
Domain Layer
IPedidoRepository (interface)
Pedido (entity)

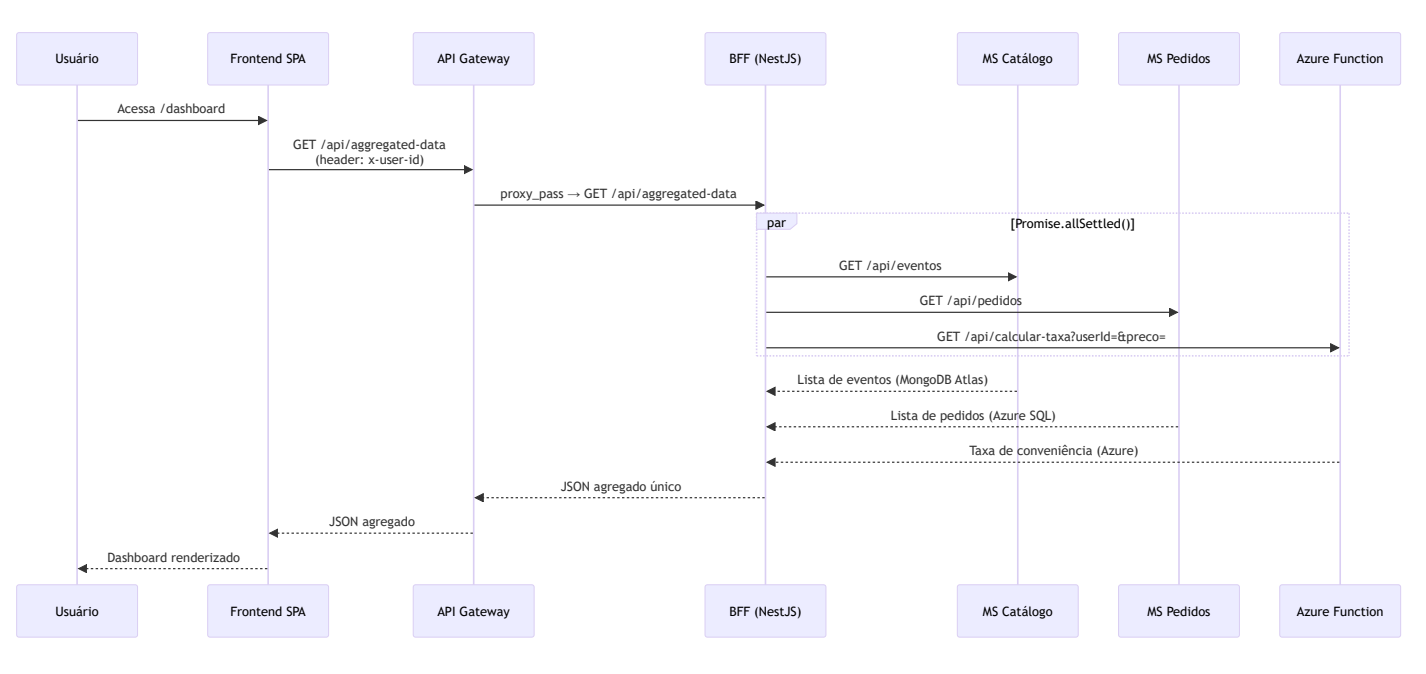
5.7 Diagrama de Classes — Entidades de Domínio


«interface»
IEventoRepository
+GetAllAsync() : Task<List<Evento>>
+GetByIdAsync(id) : Task<Evento>
+CreateAsync(evento) : Task<Evento>
+UpdateAsync(id, evento) : Task
+DeleteAsync(id) : Task
Evento
+string Id
+string Nome
+DateTime Data
+string Local
+int Capacidade
+decimal PrecoBase
+string Categoria
+string Status
«interface»
IPedidoRepository
+GetAllAsync() : Task<List<Pedido>>
+GetByIdAsync(id) : Task<Pedido>
+CreateAsync(pedido) : Task<Pedido>
+UpdateAsync(pedido) : Task
+DeleteAsync(id) : Task
Pedido
+int Id
+string EventoId
+string ClienteNome
+string ClienteEmail
+int Quantidade
+decimal ValorTotal
+string Status
+DateTime DataPedido

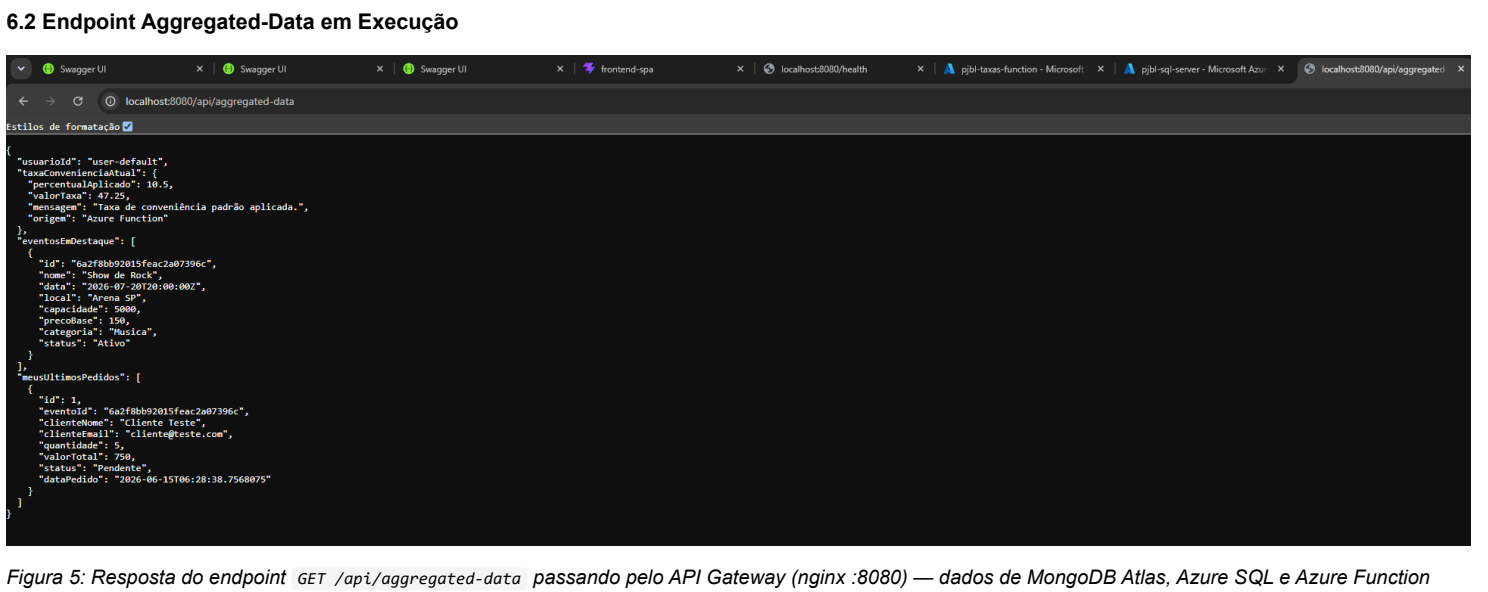
---

6. Visão de Runtime

6.1 Cenário: Usuário Acessa o Dashboard (Fluxo Principal)


Usuário
Frontend SPA
API Gateway
BFF (NestJS)
MS Catálogo
MS Pedidos
Azure Function
Acessa /dashboard
GET /api/aggregated-data (header: x-user-id)
proxy_pass → GET /api/aggregated-data
par
[Promise.allSettled()]
GET /api/eventos
GET /api/pedidos
GET /api/calcular-taxa?userId=&preco=
Lista de eventos (MongoDB Atlas)
Lista de pedidos (Azure SQL)
Taxa de conveniência (Azure)
JSON agregado único
JSON agregado
Dashboard renderizado

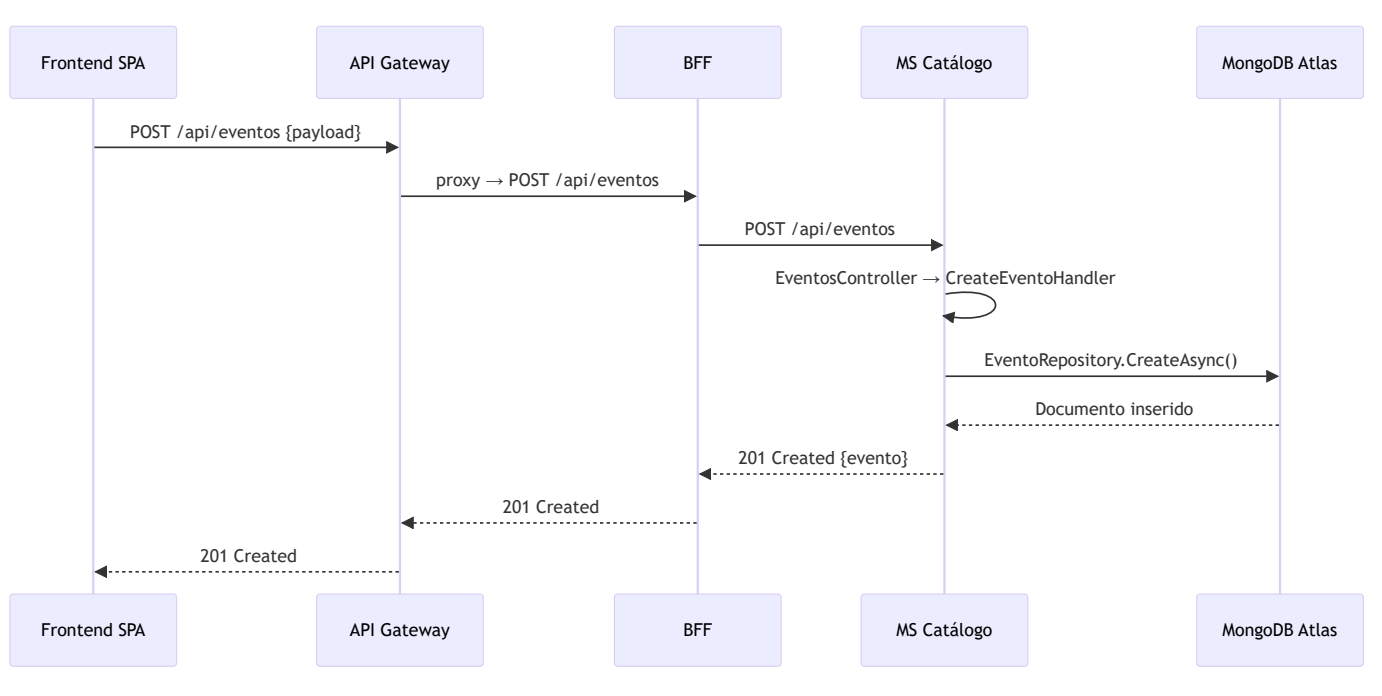
6.2 Endpoint Aggregated-Data em Execução


localhost:8080/api/aggregated-data

```
Estilos de formatação ☑
{
  "usuarioId": "user-default",
  "taxaConvenienciaAtual": {
    "percentualAplicado": 10.5,
    "valorTaxa": 47.25,
    "mensagem": "Taxa de conveniência padrão aplicada.",
    "origem": "Azure Function"
  },
  "eventosEmDestaque": [
    {
      "id": "6a2f8bb92015feac2a07396c",
      "nome": "Show de Rock",
      "data": "2026-07-20T20:00:00Z",
      "local": "Arena SP",
      "capacidade": 5000,
      "precoBase": 150,
      "categoria": "Musica",
      "status": "Ativo"
    }
  ],
  "meusUltimosPedidos": [
    {
      "id": 1,
      "eventoId": "6a2f8bb92015feac2a07396c",
      "clienteNome": "Cliente Teste",
      "clienteEmail": "cliente@teste.com",
      "quantidade": 5,
      "valorTotal": 750,
      "status": "Pendente",
      "dataPedido": "2026-06-15T06:28:38.7568075"
    }
  ]
}
```

Figura 5: Resposta do endpoint `GET /api/aggregated-data` passando pelo API Gateway (nginx :8080) — dados de MongoDB Atlas, Azure SQL e Azure Function agregados em um único JSON.

6.3 Cenário: Criação de Evento

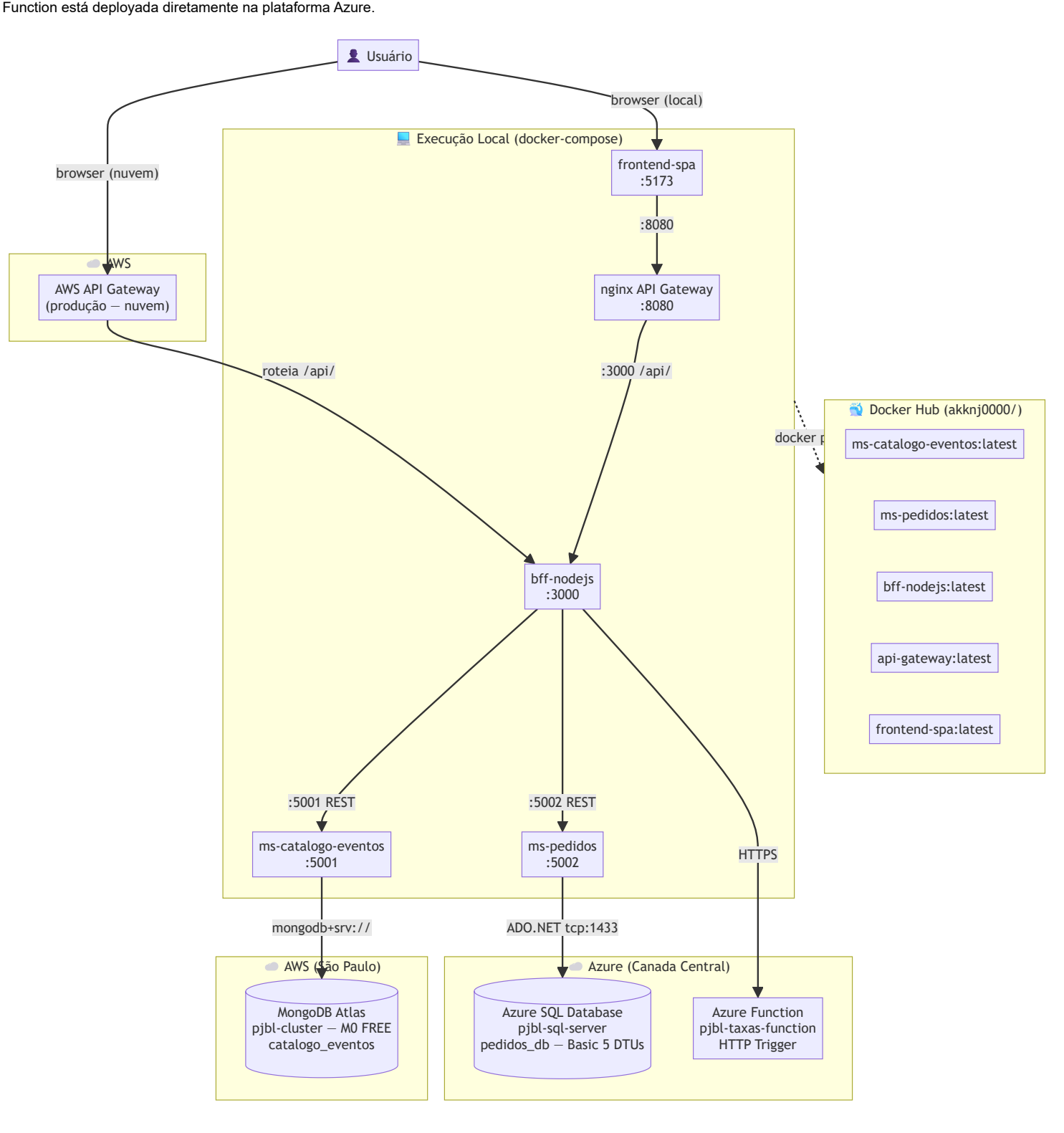

Frontend SPA
API Gateway
BFF
MS Catálogo
MongoDB Atlas
POST /api/eventos {payload}
proxy → POST /api/eventos
POST /api/eventos
EventosController → CreateEventoHandler
EventoRepository.CreateAsync()
Documento inserido
201 Created {evento}
201 Created
201 Created

---

7. Visão de Deployment

7.1 Infraestrutura — Configuração Atual

O sistema utiliza **persistência em nuvem gerenciada** (MongoDB Atlas + Azure SQL) combinada com **containers Docker** para os serviços de aplicação. A Azure Function está deployada diretamente na plataforma Azure.


Usuário
browser (nuvem)
browser (local)
Execução Local (docker-compose)
frontend-spa :5173
:8080
nginx API Gateway :8080
AWS
AWS API Gateway (produção — nuvem)
roteia /api/
:3000 /api/
docker push
Docker Hub (akknj0000/)
ms-catalogo-eventos:latest
ms-pedidos:latest
bff-nodejs:latest
api-gateway:latest
frontend-spa:latest
bff-nodejs :3000
:5001 REST
:5002 REST
HTTPS
ms-catalogo-eventos :5001
ms-pedidos :5002
mongodb+srv://
ADO.NET tcp:1433
AWS (São Paulo)
MongoDB Atlas pjbl-cluster — M0 FREE catalogo_eventos
Azure (Canada Central)
Azure SQL Database pjbl-sql-server pedidos_db — Basic 5 DTUs
Azure Function pjbl-taxas-function HTTP Trigger

7.2 Variáveis de Ambiente

Configuradas via arquivo `.env` (não versionado) referenciado no `docker-compose.yml` :

```
MONGODB_CONNECTION_STRING=mongodb+srv://<user>:<password>@pjbl-cluster.mongodb.net/catalogo_eventos
AZURE_SQL_CONNECTION_STRING=Server=tcp:pjbl-sql-server.database.windows.net,1433;...
AZURE_FUNCTION_URL=https://pjbl-taxas-function-dhbcg8bjhtbafya5.canadacentral-01.azurewebsites.net
```

7.3 Imagens Docker Hub

Imagem	Tag	Docker Hub
MS Catálogo de Eventos	latest	`akknj0000/ms-catalogo-eventos:latest`
MS Pedidos	latest	`akknj0000/ms-pedidos:latest`
BFF NestJS	latest	`akknj0000/bff-nodejs:latest`
API Gateway (nginx)	latest	`akknj0000/api-gateway:latest`
Frontend SPA	latest	`akknj0000/frontend-spa:latest`

7.4 Azure Function Deployada

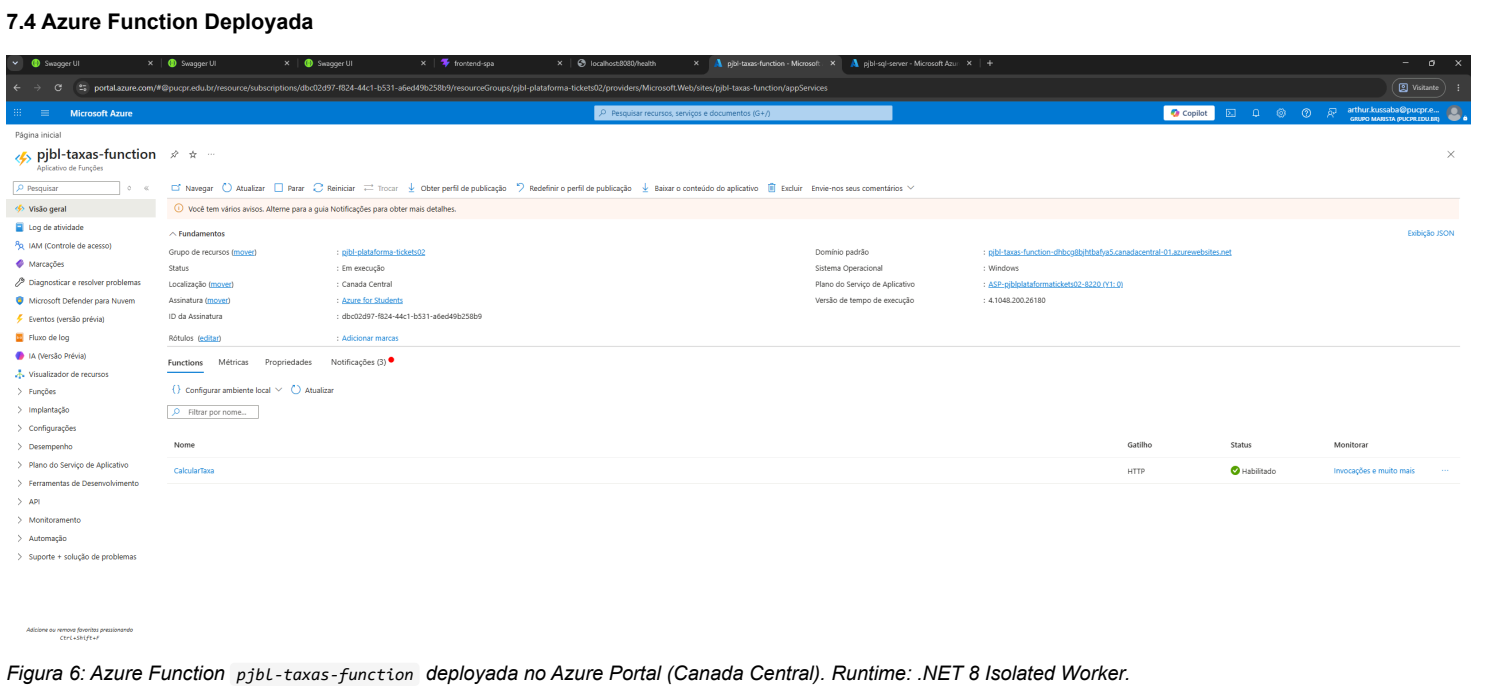

Microsoft Azure
pjbl-taxas-function

Figura 6: Azure Function `pjbl-taxas-function` deployada no Azure Portal (Canada Central). Runtime: .NET 8 Isolated Worker.

7.5 MongoDB Atlas — Cluster

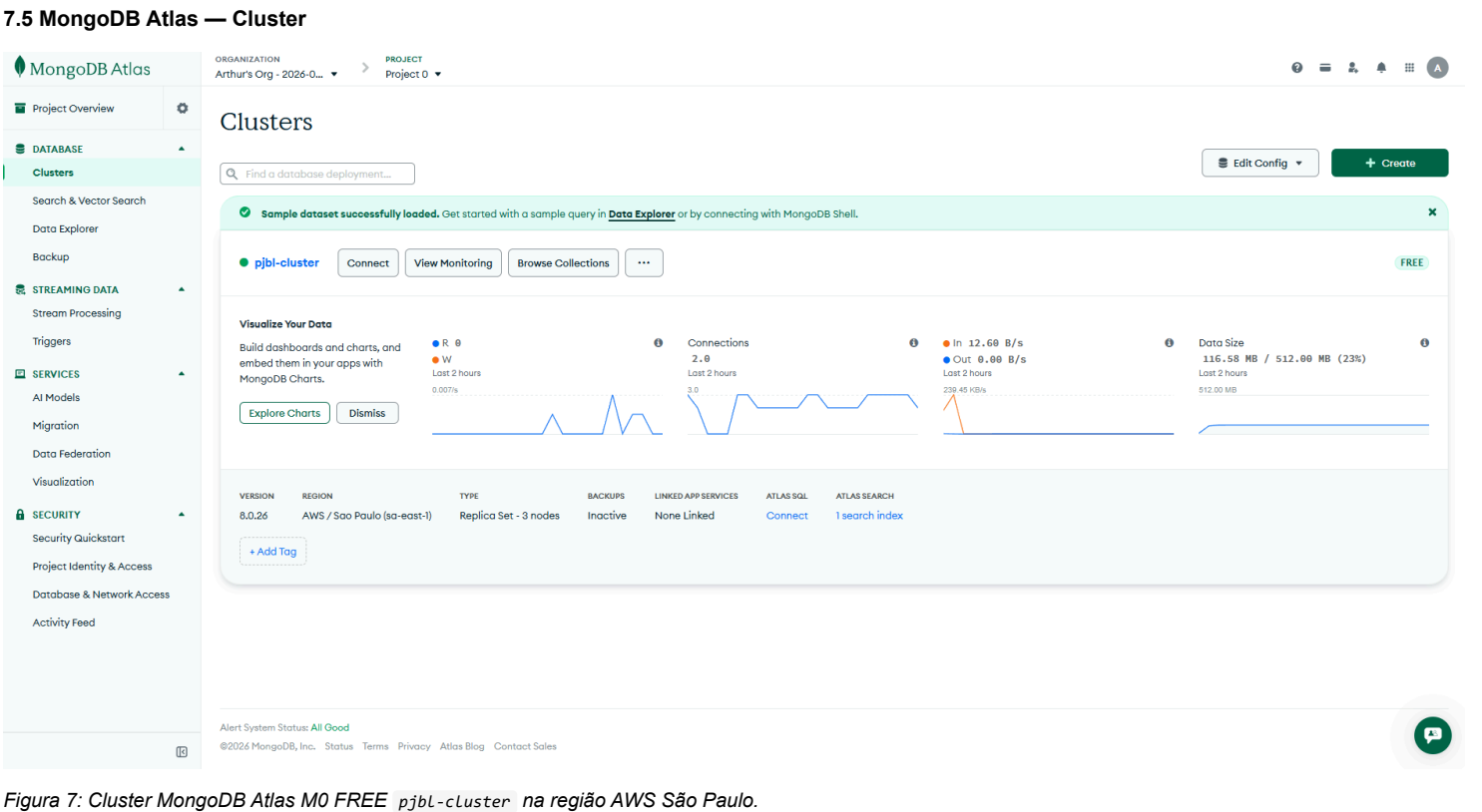

MongoDB Atlas
Clusters
pjbl-cluster
FREE
AWS / Sao Paulo (sa-east-1)

Figura 7: Cluster MongoDB Atlas M0 FREE `pjbl-cluster` na região AWS São Paulo.

7.6 MongoDB Atlas — Collections

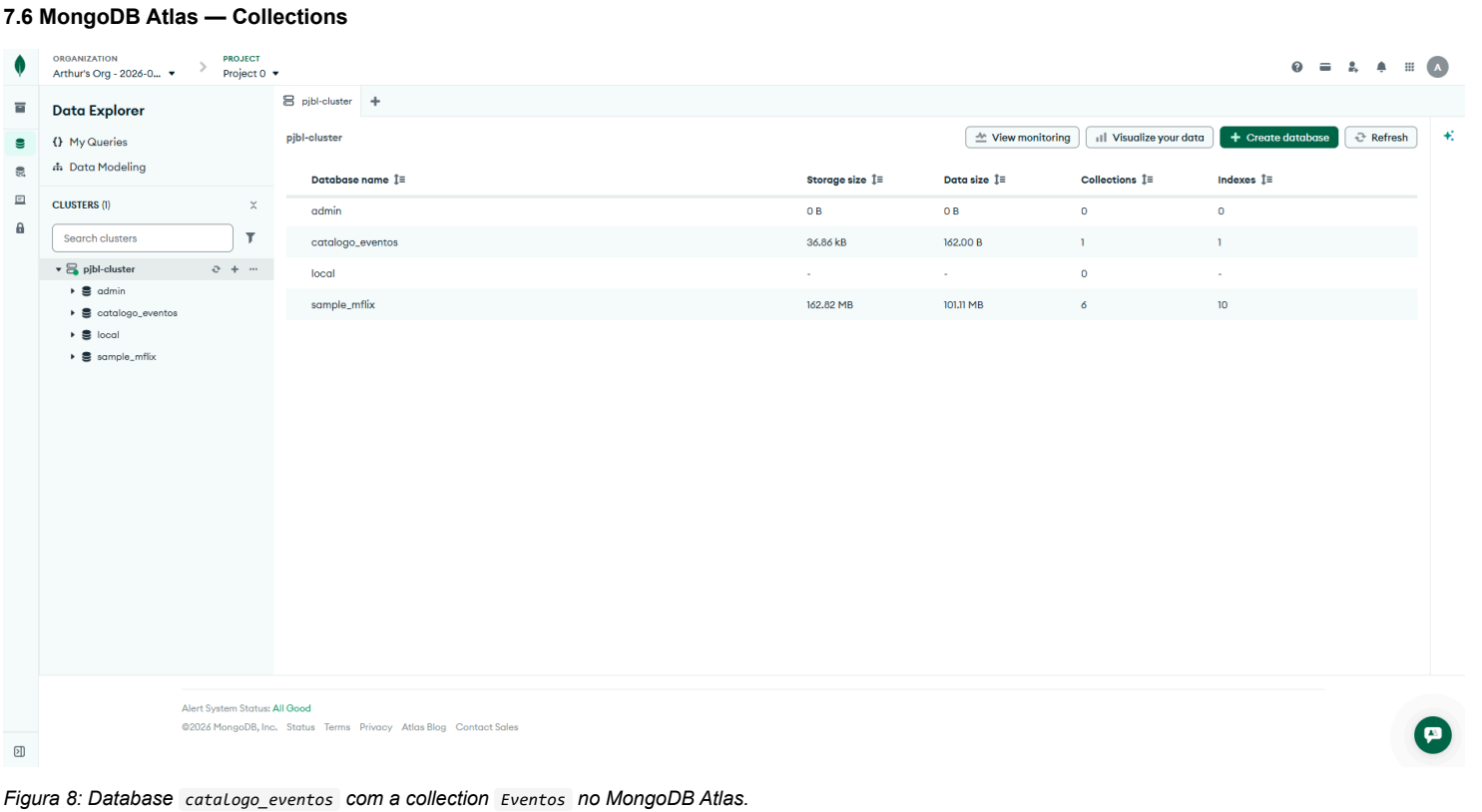

Data Explorer
pjbl-cluster
admin
catalogo_eventos
local
sample_mflix

Figura 8: Database `catalogo_eventos` com a collection `Eventos` no MongoDB Atlas.

7.7 Azure SQL Database

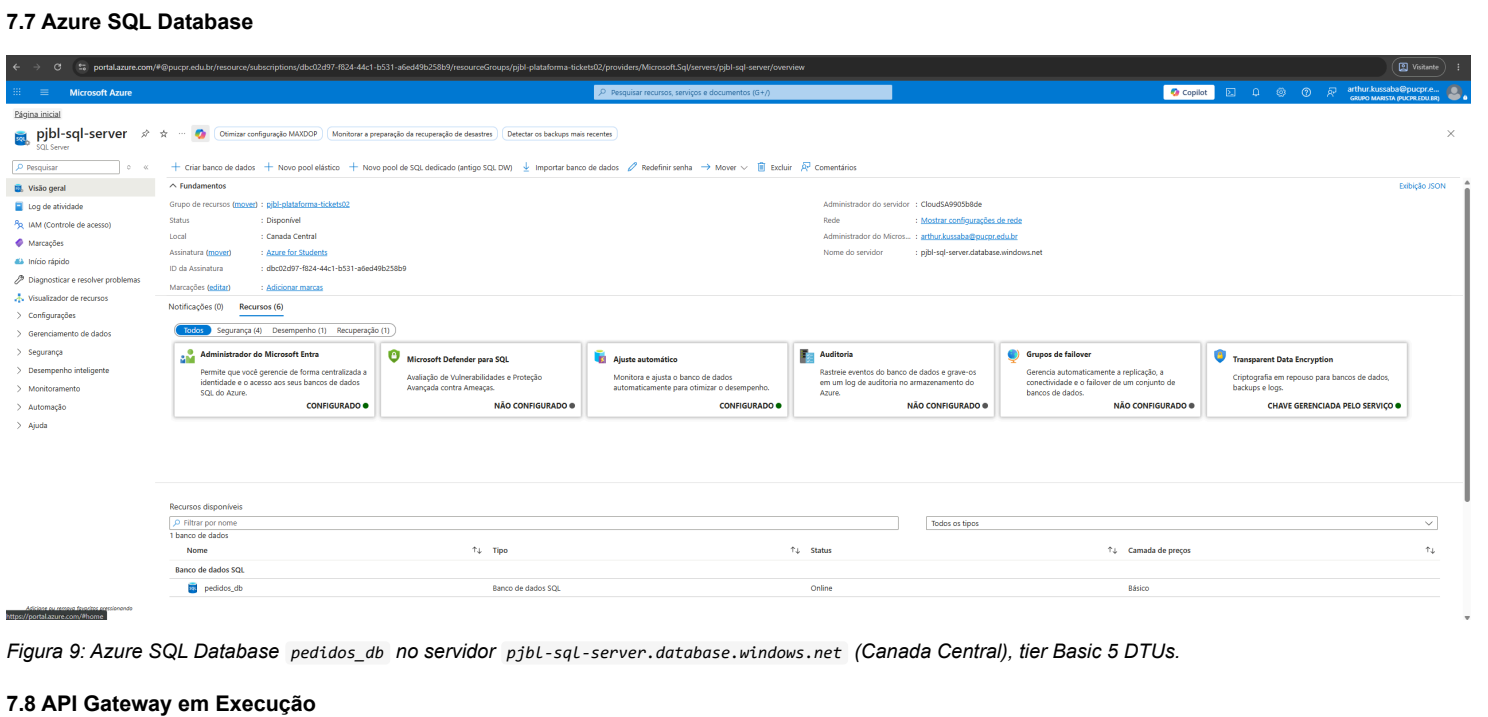

Microsoft Azure
pjbl-sql-server
pedidos_db

Figura 9: Azure SQL Database `pedidos_db` no servidor `pjbl-sql-server.database.windows.net` (Canada Central), tier Basic 5 DTUs.

7.8 API Gateway em Execução

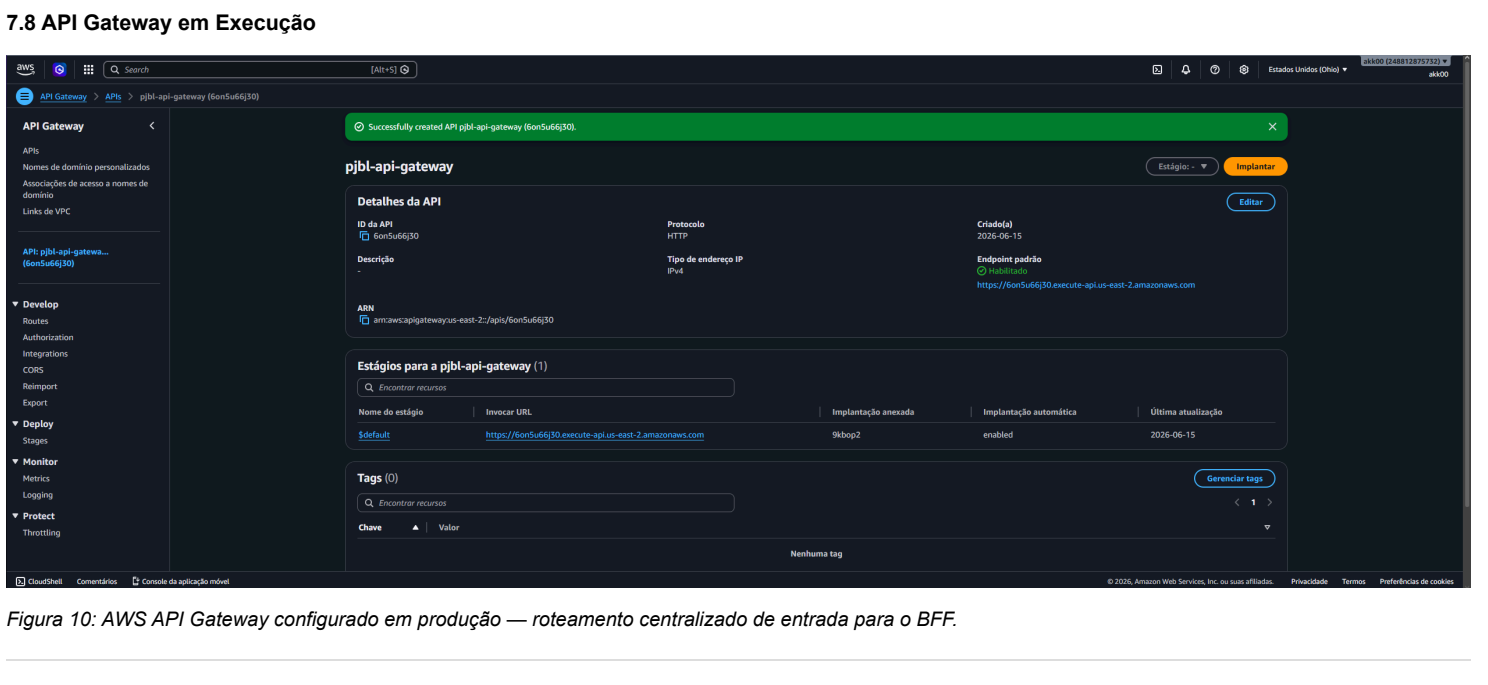

API Gateway
pjbl-api-gateway
Detalhes da API
Estágios para a pjbl-api-gateway (1)
$default

Figura 10: AWS API Gateway configurado em produção — roteamento centralizado de entrada para o BFF.

---

8. Conceitos Transversais

8.1 Modelo de Domínio

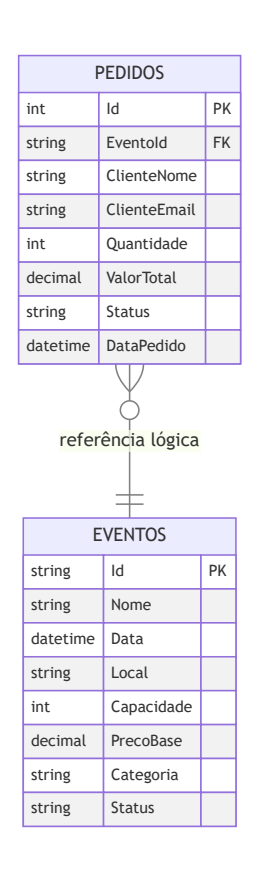
Evento (MongoDB Atlas — collection `Eventos`):

- `Id` (ObjectId), `Nome` , `Data` , `Local` , `Capacidade` , `PrecoBase` , `Categoria` , `Status`

Pedido (Azure SQL — tabela `Pedidos`):

- `Id` (int, PK), `EventoId` (referência lógica), `ClienteNome` , `ClienteEmail` , `Quantidade` , `ValorTotal` , `Status` , `DataPedido`

8.2 Diagrama Entidade-Relacionamento


PEDIDOS
int Id PK
string EventoId FK
string ClienteNome
string ClienteEmail
int Quantidade
decimal ValorTotal
string Status
datetime DataPedido
referência lógica
EVENTOS
string Id PK
string Nome
datetime Data
string Local
int Capacidade
decimal PrecoBase
string Categoria
string Status

> Nota: `EventoId` em Pedidos é uma referência lógica (string), não uma FK de banco cruzado. Os bancos são independentes.

8.3 Segurança

- Nível acadêmico: sem autenticação JWT neste escopo
- Senhas de bancos gerenciadas via variáveis de ambiente (`.env` não versionado)
- Credenciais do Azure SQL e MongoDB Atlas nunca expostas no repositório

8.4 Logging e Observabilidade

- ASP.NET Core: logs padrão via `ILogger<T>`
- NestJS: `Logger` nativo do NestJS em cada módulo
- Azure Function: logs via `ILogger` do isolated worker

8.5 Tratamento de Erros

- MS C#: `404 NotFound` para recursos inexistentes, `400 BadRequest` para validação
- BFF: `Promise.allSettled()` com fallback — nunca retorna 500 por falha de dependência
- Azure Function: retorna taxa padrão `10.5%` para parâmetros inválidos

8.6 Testes

- **Unitários**: xUnit + Moq — testam handlers de Application isolados de Infrastructure
- **Arquitetura**: NetArchTest.Rules — garantem que Domain não depende de Infrastructure
- **Execução**: `dotnet test` em cada solução

---

9. Decisões de Arquitetura (ADRs)

ADR-001: Vertical Slice dentro de Clean Architecture

Contexto: Clean Architecture tradicional organiza código por camada técnica. Para features distintas, isso aumenta acoplamento entre funcionalidades não relacionadas.

Decisão: Vertical Slice dentro da camada Application. Cada feature (`CreateEvento` , `GetEvento` , etc.) tem seu próprio diretório com Handler, Request e Response.

Consequência: Adicionar nova feature exige tocar apenas um diretório; sem risco de quebrar outras features.

ADR-002: Persistência Poliglota — MongoDB Atlas + Azure SQL

Contexto: Eventos têm schema flexível; pedidos exigem consistência transacional.

Decisão: MongoDB Atlas M0 FREE para MS Catálogo (schema livre, documentos JSON), Azure SQL Basic para MS Pedidos (ACID, schema rígido). Bancos gerenciados em cloud — sem containers de banco no docker-compose.

Consequência: Cada MS é dono exclusivo de seu banco. Sem joins entre bancos. `EventoId` no Pedido é referência lógica.

ADR-003: BFF com Promise.allSettled()

Contexto: Frontend precisa de dados de 3 fontes. Chamadas sequenciais aumentam latência; falha em cascata derrubaria o dashboard.

Decisão: BFF realiza 3 chamadas em paralelo com `Promise.allSettled()` . Falhas retornam fallback.

Consequência: Dashboard renderiza parcialmente mesmo com um serviço fora do ar.

ADR-004: EnsureCreated() em vez de Migrations EF Core

Contexto: EF Core Migrations exigem histórico controlado, overhead desnecessário em escopo acadêmico.

Decisão: `EnsureCreated()` na inicialização do MS Pedidos. Azure SQL cria o schema automaticamente.

Consequência: Schema não é versionado; adequado para o contexto de laboratório.

ADR-005: Azure Function Deployed em Azure (não em container)

Contexto: Azure Functions em container Docker exigem `AzureWebJobsStorage` configurado. Em ambiente acadêmico, provisionar Azure Storage adiciona complexidade.

Decisão: Deploy da Azure Function diretamente no Azure (Canada Central) via Kudu zip deploy. O BFF consome via HTTPS.

Consequência: A função não roda localmente via Docker. URL de produção hardcoded como fallback no BFF.

ADR-006: Monorepo em vez de 5 repositórios separados

Contexto: Requisito lista 5 componentes. Manutenção de 5 repos com cross-cutting concerns (docker-compose, CI/CD, docs) aumenta overhead sem benefício acadêmico.

Decisão: Único monorepo público com todos os componentes, estruturado por pasta.

Consequência: Um único `git clone` entrega todo o sistema. docker-compose.yml na raiz orquestra tudo.

ADR-007: AWS API Gateway em produção + nginx localmente

Contexto: O sistema precisa de um API Gateway para centralizar entrada e rotear tráfego para o BFF. Duas opções viáveis: nginx (simples, containerizável) e AWS API Gateway (gerenciado, escalável, com throttling nativo).

Decisão: nginx como API Gateway no ambiente local (docker-compose), pois é containerizável e integra naturalmente ao `pjbl-network` . AWS API Gateway como gateway de produção na nuvem, substituindo o nginx e oferecendo roteamento gerenciado, logs no CloudWatch e escalabilidade automática sem infraestrutura própria.

Consequência: O ambiente local reproduz o comportamento de gateway via nginx. O ambiente de nuvem utiliza AWS API Gateway apontando para o BFF. Ambos preservam a abstração da topologia interna dos microsserviços para o cliente.

---

10. Riscos e Débitos Técnicos

Risco	Impacto	Mitigação
Azure Function não roda em container local	Médio	URL de produção no Azure como fallback no BFF
Acoplamento temporal entre serviços no docker-compose	Médio	`depends_on` garante ordem de inicialização
Ausência de autenticação JWT	Médio	Escopo acadêmico; produção exigiria OAuth2/OIDC
Sem circuit breaker no BFF	Baixo	`Promise.allSettled()` com fallback mitiga
Azure SQL Basic 5 DTUs — throttling em carga alta	Baixo	Nível acadêmico; produção usaria Standard+
MongoDB Atlas M0 — limite de conexões simultâneas	Baixo	Adequado para demo; produção usaria tier pago
Imagens Docker com `latest` tag	Baixo	Em produção, usar tags versionadas para determinismo

---

11. Glossário

Termo	Definição
BFF	Backend for Frontend — camada de agregação dedicada ao frontend
Clean Architecture	Padrão de Robert C. Martin — isolamento entre Domain, Application, Infrastructure e API
Vertical Slice	Organização de código por feature, não por camada técnica
Microsserviço	Serviço independente com domínio, banco e deploy próprios
Persistência Poliglota	Uso de diferentes tecnologias de banco de dados conforme o domínio
Azure Function	Função serverless executada via HTTP Trigger na plataforma Azure
Isolated Worker	Modelo de execução do Azure Functions para .NET que roda fora do processo host
EnsureCreated()	Método do EF Core que cria o banco se não existir, sem controle de migrations
Promise.allSettled()	Método JavaScript que executa N promises em paralelo e retorna todos os resultados, incluindo falhas
ObjectId	Tipo de ID padrão do MongoDB, gerado automaticamente pelo driver
ADR	Architecture Decision Record — registro formal de decisão arquitetural
C4 Model	Modelo de diagramação hierárquica (Context → Container → Component → Code)
EDA	Event-Driven Architecture — arquitetura orientada a eventos
Kudu	Serviço de deploy do Azure App Service / Azure Functions via zip deploy
DTU	Database Transaction Unit — métrica de capacidade do Azure SQL